

ApfelContext — Everything Explained

A complete guide to what this project is, how it works, what every technology does, and a word-by-word breakdown of the three bullet points that describe it.

Table of Contents

1. What Problem This Solves
2. Architecture — The Big Picture
3. Tech Stack — Every Tool Explained
4. How The Code Is Organized
5. Key Concepts Explained Simply
6. Bullet Point 1 — Word by Word
7. Bullet Point 2 — Word by Word
8. Bullet Point 3 — Word by Word

1. What Problem This Solves

When you use Claude Code for a long time, the conversation gets very long. Claude has a **context window** — a maximum amount of text it can hold in memory at once (like working memory in your brain). When the conversation fills that window, Claude has to **compact** the old messages — squish them down into a shorter summary so new messages fit.

The problem: Claude normally does this compaction itself, using its own API. That costs real money — every token (roughly every 4 characters) that Claude reads or writes is billed. Compaction can burn thousands of tokens per session just on housekeeping.

The solution this project builds: Instead of letting Claude compact its own context, intercept that work and send it to a **free, local AI model** running on your own Mac. The local model does the summarization and compression, Claude gets back a short result, and you paid nothing.

This is ApfelContext. The name "Apfel" is German for "Apple" — it's built for Apple Silicon Macs (the M-series chips).

2. Architecture — The Big Picture

Claude Code (the AI assistant you talk to)

```
When you type a message → UserPromptSubmit hook fires
When Claude edits a file → PostToolUse hook fires
When Claude finishes responding → Stop hook fires
When a new session starts → SessionStart hook fires
```

```
stdio (text in/out over a pipe)
```

▼

MCP Server (src/server.py)

Exposes 7 tools Claude can call:

```
compact_prompt  log_event  summarize_history
generate_handoff get_context get_info  set_model
```

HTTP (OpenAI-compatible API format)

▼

Ollama (running locally on port 11434)

Hosts the Gemma 4 E4B model

Runs entirely on your Mac – no internet required

Also running in parallel:

Hook Daemon (scripts/hook_runner.py --serve)

Tiny HTTP server on port 7737

Receives commands from shell hooks without Python startup cost

Managed by launchd (macOS's background job manager)

State is saved in:

state/ directory

```
progress.md    – what happened this session
bug.md         – bugs encountered
decision.md    – architectural decisions made
architecture.md – structural notes
codebase.md    – live map of every file and what it does
```

Data flow in plain English:

1. You type a message to Claude.
2. Before Claude sees it, a hook intercepts it, sends it to Gemma on your Mac, and gets back a shorter version. Claude receives the compressed version.
3. Claude decides it needs to call a tool (e.g., `compact_prompt`). It sends a JSON message over stdio to the MCP server.
4. The MCP server receives the call, runs the function, and sends the result back over stdio.
5. If the function needs to summarize text, it calls Ollama over HTTP — same format as the OpenAI API, but pointing at localhost instead of the internet.
6. State (what happened, what files changed) is written to `.md` files in `state/`.
7. At the next session start, those state files are injected back into Claude's context so it remembers what it was doing.

3. Tech Stack — Every Tool Explained

Python

The entire project is written in Python. Python is the most common language for AI tooling because:

- It has libraries for almost every AI/ML task.
- It is easy to read and write.
- It starts up fast enough for short-lived scripts (the hooks).

Version used: Python 3.12+ (required because the code uses `str | None` type hints, which only work in newer Python).

FastMCP

What it is: A Python library that implements the Model Context Protocol server for you.

What MCP is: A standard protocol (like a contract) that defines how AI assistants and external tools communicate. Claude Code speaks MCP; your server must also speak MCP for Claude to talk to it.

What FastMCP does for you: Without FastMCP, you would have to manually parse and write JSON-RPC messages (a specific format for sending function calls over text). FastMCP hides all of that. You just write a normal Python function, put `@mcp.tool` above it, and FastMCP:

- Registers the function as a tool Claude can discover.
- Reads the type hints (`text: str, token_budget: int = 2000`) and turns them into a JSON schema that tells Claude what arguments the tool accepts.
- Reads the docstring and uses it as the tool's description (so Claude knows when to call it).
- Routes incoming calls to the right function.

Transport: stdio. FastMCP reads from `sys.stdin` and writes to `sys.stdout`. This is why there is a hard rule: **never use `print()` anywhere in `src/`** — printing to stdout would corrupt the MCP protocol messages.

Ollama

What it is: A program that runs AI language models locally on your computer. Think of it as a small server that hosts an AI model and gives it an API you can call.

How it works: Ollama downloads the model weights (the numbers that define how the model thinks) and loads them into memory. It then listens on `http://localhost:11434` for requests. When you send it text, it runs the model and sends back a response.

Why it matters here: Ollama means you never send your code or conversation to any company's server when doing compression/summarization. Everything stays on your Mac.

OpenAI-compatible API: Ollama's API is designed to look exactly like OpenAI's API. This means you can use the official `openai` Python SDK and just change the base URL from `https://api.openai.com/v1` to `http://localhost:11434/v1`. The code in `apfel_client.py` does exactly this.

Gemma 4 E4B (the model)

What it is: A language model made by Google, designed to run efficiently on consumer hardware.

"Gemma 4" — the 4th generation of the Gemma model family.

"E4B" — stands for "Expert 4 Billion" — it is a Mixture-of-Experts (MoE) model. MoE means the model has more than 4 billion parameters total, but only activates a subset (~4 billion worth) per token. This makes it fast while still being capable.

Why Gemma 4 E4B specifically: It was benchmarked against another model (qwen2.5:1.5b) and produced better output for structured tasks. It runs at 86 tokens/second on Apple Silicon M-series chips, which is fast enough to compress a prompt in under 2 seconds.

Context window: 128,000 tokens — large enough to handle entire conversation histories.

Apple Silicon / Metal GPU

What it is: Apple M1/M2/M3/M4 chips have a built-in GPU that shares memory with the CPU (called "unified memory"). Ollama uses Apple's Metal framework to run the model on this GPU.

Why it matters: Running a 4-billion-parameter model on a CPU alone would be too slow. The Metal GPU backend makes it fast enough to be useful in real time (86 tok/s).

OpenAI Python SDK

What it is: The official Python library for calling OpenAI's API. It handles HTTP requests, retries, timeouts, and parsing responses.

Why used here (not with OpenAI): Because Ollama's API is OpenAI-compatible, you can use the exact same SDK. `ApfelClient` in `apfel_client.py` creates an `OpenAI` client object but points it at `http://localhost:11434/v1` instead of OpenAI's servers. The `api_key` is set to `"unused"` — a non-empty placeholder required by the SDK, even though Ollama doesn't check it.

Claude Code Hooks

What they are: Shell scripts that Claude Code runs automatically at specific moments in its lifecycle. You register them in `.claude/settings.json`.

The four hooks in this project:

Hook	When it fires	What it does
<code>SessionStart</code>	When a new Claude session opens	Injects state files into context, warms up Gemma
<code>UserPromptSubmit</code>	Just before Claude sees your message	Compresses your message with Gemma if it's long prose
<code>PostToolUse</code>	After Claude edits/writes a file	Logs the edit, updates the codebase map

Hook	When it fires	What it does
Stop	When Claude finishes its response	Marks the turn as complete if files were changed

Each hook is a shell script (`.claude/hooks/on-*.sh`) that calls `hook_runner.py`, which does the actual work.

launchd

What it is: macOS's built-in background job manager. Similar to `systemd` on Linux or Windows Services on Windows.

How it's used here: `hook_runner.py --serve` starts a tiny HTTP daemon on port 7737. Instead of launching a new Python process for every hook event (which takes ~200ms each time), the hooks send HTTP requests to this already-running daemon. launchd ensures the daemon starts automatically at login and restarts it if it crashes.

The launchd configuration is in `scripts/com.apfel-context.daemon.plist` — a `.plist` is an XML configuration file that macOS uses.

tiktoken

What it is: OpenAI's tokenizer library. A tokenizer converts text into "tokens" — the units that language models count. One token is roughly 4 characters (or 3/4 of a word).

How it's used: In `hook_runner.py`, when compressing a user's prompt, tiktoken counts the exact tokens before and after compression so the tool can report "reduced from 300 to 140 tokens (-53%)".

fcntl (file locking)

What it is: A low-level Unix system call for locking files. `fcntl.flock()` gives your process exclusive access to a file so other processes can't write to it at the same time.

Why needed: The hooks run as separate processes. Multiple hooks could fire simultaneously (e.g., `PostToolUse` and `Stop` overlap). Without locking, two processes writing to `progress.md` at the same time could corrupt it (interleaved characters). `StateManager` uses a "sidecar lock file" — a separate `.lock` file — so the lock doesn't interfere with opening the actual state file in append mode.

pytest

What it is: Python's most popular testing framework. You write functions that start with `test_`, and pytest runs them all and reports which pass or fail.

Scale here: 138 tests (105 were written in session 6-7; additional tests added later). They cover `StateManager`, `ApfelClient`, `server.py` tools, hooks, config, and an integration suite that runs against a real Ollama instance.

4. How The Code Is Organized

```

apfel-compactor/
├── src/                                ← Core Python package
│   ├── server.py                      ← MCP server: the 7 tools Claude can call
│   ├── apfel_client.py                ← Wrapper around Ollama (LLM calls)
│   ├── state_manager.py              ← Read/write state/*.md files safely
│   ├── chunker.py                    ← Split large text into model-sized pieces
│   ├── config.py                     ← Read config.json + env vars
│   └── health.py                     ← Check if Ollama is running
├── scripts/
│   ├── hook_runner.py                ← CLI for hooks; also runs as HTTP daemon
│   ├── install.sh                    ← Global installer
│   ├── uninstall.sh                  ← Removes everything
│   └── com.apfel-context.daemon.plist ← launchd config
├── .claude/
│   └── hooks/
│       ├── on-session-start.sh       ← SessionStart hook
│       ├── on-edit.sh                ← PostToolUse hook
│       ├── on-stop.sh                ← Stop hook
│       └── on-prompt.sh              ← UserPromptSubmit hook
├── prompts/                          ← System prompt text files for the LLM
│   ├── compress.txt                  ← Instructions for compressing text
│   ├── summarize.txt                 ← Instructions for summarizing text
│   ├── classify.txt                   ← Instructions for categorizing an event
│   └── file_summary.txt               ← Instructions for describing a file
├── state/                            ← Runtime state (not committed to git)
│   ├── progress.md
│   ├── bug.md
│   ├── decision.md
│   ├── architecture.md
│   └── codebase.md
├── tests/                            ← 138 tests
│   ├── test_state_manager.py
│   ├── test_server.py
│   ├── test_config.py
│   ├── test_hook_runner.py
│   └── test_integration.py

```

5. Key Concepts Explained Simply

What is a Token?

A token is the basic unit a language model works with. It is roughly:

- 1 short word = 1 token ("cat", "run", "the")
- 1 long word = 2–3 tokens ("uncomfortable" = 3)
- 4 characters $\approx$ 1 token on average

When people say "GPT-4 has a 128K context window," they mean it can hold 128,000 tokens in memory at once — about 100,000 words, or a short novel.

What is Compaction / Compression?

Compaction means taking a long text and rewriting it shorter while keeping the meaning. Not just cutting words — an AI does it intelligently, like a good summarizer.

Example:

- **Before (20 tokens):** "I would really appreciate it if you could possibly take a look at this file and let me know what you think about the code quality."
- **After (9 tokens):** "Please review this file's code quality."

What is Stdio?

Stdio means "standard input / standard output." Every program on your computer has:

- **stdin** — a pipe it reads input from (usually your keyboard)
- **stdout** — a pipe it writes output to (usually your terminal)
- **stderr** — a second output pipe for errors/logs

When Claude Code launches the MCP server, it connects Claude's output to the server's stdin, and the server's stdout back to Claude. This is how they communicate — by reading and writing text in JSON format over these pipes. It's low-tech but reliable.

What is JSON-RPC?

JSON-RPC is a protocol for calling functions over text. A call looks like this:

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "method": "tools/call",
  "params": {
    "name": "compact_prompt",
    "arguments": {"text": "Please review the code quality..."}
  }
}
```

And the response looks like:

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "result": {"content": [{"type": "text", "text": "Review code
quality..."}]}
}
```

FastMCP handles all of this format automatically.

What is Path Traversal?

Path traversal is a security attack where a user supplies a filename like `../../etc/passwd` (going "up" directories with `..`) to trick a program into accessing files it shouldn't. In this project, `event_type` is a user-supplied string used to build a file path like `state/{event_type}.md`. If someone passed `../../../secret`, it could write to a file outside the `state/` directory. The three sanitization layers prevent this:

1. **Regex strip** — removes everything that isn't a lowercase letter: `../../etc` → `etcpasswd`
2. **Frozenset check** — only `{"progress", "bug", "decision", "architecture"}` are valid
3. **is_relative_to() check** — after building the full path, verify it's still inside `state/`

What is O(1)?

Big-O notation describes how an algorithm scales. $O(1)$ means "constant time" — the operation takes the same amount of time no matter how large the input is.

In this context: the session-start context injection is **$O(1)$** because it always produces at most 400 tokens of output, regardless of whether the state files are 1KB or 100KB. It compresses large state files down to fit the budget, so it never grows.

Compare to $O(n)$, which would mean "the bigger the state files, the more tokens injected" — that would eventually overflow Claude's context window.

6. Bullet Point 1 — Word by Word

"Built a local MCP server in Python/FastMCP that routes Claude Code's context compaction through Gemma 4 E4B on Ollama, cutting 53-70% of tokens at 86 tok/s on Apple Silicon while eliminating API spend on compression and keeping source code off remote servers"

"Built a local MCP server" Created a program that runs on your own computer (local = not on the internet) that implements the Model Context Protocol (MCP) — the standard that lets Claude call external tools. A "server" here just means a program that listens for requests and responds to them.

"in Python/FastMCP" The server is written in Python and uses the FastMCP library, which handles all the protocol wiring automatically (see Section 3).

"that routes Claude Code's context compaction" "Routes" means it intercepts and redirects. Claude Code normally does context compaction itself (using Claude's own API). This server intercepts those

compaction requests and sends them somewhere else instead. Context compaction = the process of summarizing old conversation history when the context window gets full.

"through Gemma 4 E4B on Ollama" Sends the compaction work to Gemma 4 E4B (a Google AI model, the E4B variant) running inside Ollama (a local model runner). Instead of Claude summarizing itself, Gemma does it for free on your Mac.

"cutting 53-70% of tokens" The output from compression is 53% to 70% shorter (in token count) than the input. If you sent in 1000 tokens, you get back 300–470 tokens. This is the measured compression ratio across real prompts during development.

"at 86 tok/s" Gemma 4 E4B generates 86 tokens per second on Apple Silicon. "tok/s" = tokens per second. This means it is fast enough to compress a prompt in under 2 seconds in real time.

"on Apple Silicon" Specifically optimized for (and tested on) Apple's M-series chips (M1, M2, M3, M4). These chips have unified memory and a Metal GPU that Ollama uses to run models efficiently.

"while eliminating API spend on compression" "API spend" = money paid per API call to Anthropic/OpenAI. By routing compaction to a local model, you pay \$0 for the compression step. The word "eliminating" means this cost goes to zero, not just "reduced."

"and keeping source code off remote servers" When Claude compacts its context, that context includes your code, file contents, and conversation. If Claude sends it to Anthropic's servers for compaction, your code leaves your machine. With local compaction via Ollama, the text never leaves your computer — privacy is preserved.

7. Bullet Point 2 — Word by Word

"Designed O(1) session context injection capped at 400 tokens regardless of state file size, with adaptive compression budgeting and 3-layer path traversal sanitization across 7 MCP tools, validated by a 105-test suite at 100% pass rate"

"Designed O(1) session context injection" Architected (Designed) a system that, when a new Claude session starts, automatically injects project state into Claude's context. O(1) means this injection always costs the same amount — constant memory, constant time — no matter how much state has accumulated.

"capped at 400 tokens" The injection has a hard upper limit: it will never send more than 400 tokens into the context. This is enforced in `hook_runner.py` via the `max_injection_tokens` config value, defaulting to 400. This keeps session startup cheap and ensures the budget is predictable.

"regardless of state file size" Even if the `state/` files have grown to 100KB over weeks of development, the injection still produces ≤ 400 tokens. The system adapts: small state $\rightarrow$ injected verbatim; medium state $\rightarrow$ truncated to budget; large state $\rightarrow$ compressed by Gemma then hard-capped.

"with adaptive compression budgeting" "Adaptive" means the compression target adjusts based on how much input there is. The formula is `target = max(400, estimated_tokens × 0.4)`. This prevents a pathological case where you have 50,000 tokens of state and ask Gemma to compress to 400 — 99.2% loss would destroy all meaning. Instead, the floor is 40% of the original, so you always keep at least a coherent summary.

"and 3-layer path traversal sanitization" Three separate security checks prevent path traversal attacks (where a malicious `event_type` string like `../../etc/passwd` could write files outside the `state/` directory):

1. Regex strips everything that isn't a lowercase letter
2. Frozenset membership check against the four valid types
3. `Path.is_relative_to()` verifies the final resolved path stays inside `state/`

"across 7 MCP tools" The MCP server exposes exactly 7 tools Claude can call: `compact_prompt`, `log_event`, `summarize_history`, `generate_handoff`, `get_context`, `get_info`, `set_model`. The sanitization applies to any tool that touches state files.

"validated by a 105-test suite" A test suite of 105 automated tests (written using `pytest`) that verify all the above behavior actually works correctly. ("105-test suite" refers to the count at the time of that session; 138 tests exist now.)

"at 100% pass rate" All 105 tests pass. Zero failures. This means every behavior described — the O(1) cap, the adaptive budget, the sanitization — has an automated test proving it works.

8. Bullet Point 3 — Word by Word

"Engineered an async hooks pipeline (SessionStart, UserPromptSubmit, PostToolUse, Stop) that auto-compacts prompts and updates a live codebase map on each edit without blocking the agent, choosing Gemma 4 E4B over qwen2.5:1.5b after the latter mangled structured tool output in benchmarks"

"Engineered an async hooks pipeline" "Engineered" = built carefully, with attention to how it interacts with the system. "Async" = asynchronous — the hooks run without waiting for each other; they don't pause Claude while working. "Pipeline" = a series of stages that process data in sequence, each triggered by an event.

"(SessionStart, UserPromptSubmit, PostToolUse, Stop)" The four lifecycle events where hooks fire:

- **SessionStart**: when you open a new conversation with Claude
- **UserPromptSubmit**: the moment just before Claude reads your message
- **PostToolUse**: after Claude uses a tool (e.g., after writing/editing a file)
- **Stop**: when Claude finishes generating its response

"that auto-compacts prompts" Automatically compresses your messages before Claude sees them. If you write a long, wordy prompt (50+ words of prose), the `UserPromptSubmit` hook intercepts it, sends it to Gemma, and injects the compressed version. Claude reads the shorter version — saving tokens.

"and updates a live codebase map on each edit" Every time Claude edits a file (`PostToolUse` hook fires), `hook_runner.py` reads the changed file, sends it to Gemma for a one-line description, and upserts that description into `state/codebase.md`. This file is a live, always-current map of what every file in the project does — updated automatically as Claude works.

"without blocking the agent" The hooks run asynchronously in the background. Claude doesn't wait for them to finish. If Gemma takes 1.5 seconds to compress a prompt or summarize a file, Claude continues

immediately — the result is injected as context alongside the next message, not before it. This is enforced by short timeouts (8 seconds max) and exit code 0 always — hooks must never crash Claude.

"choosing Gemma 4 E4B over qwen2.5:1.5b" During development, two local models were benchmarked: Gemma 4 E4B (Google, ~4B active parameters) and qwen2.5:1.5b (Alibaba, 1.5B parameters). qwen is smaller and faster, but it was rejected.

"after the latter mangled structured tool output in benchmarks" "The latter" = qwen2.5:1.5b (the second of the two mentioned). "Mangled" = produced garbled or incorrect output. "Structured tool output" = when asked to produce specific formats like `{"type": "progress"}` for the classify tool, qwen would return malformed JSON or wrong category names. Gemma 4 E4B handled these structured tasks reliably, so it was selected.

Document generated 2026-05-19. All code references are accurate to the current state of the repository.